METHOD ESSAY · ERP-HASHDIFF-SYNC

Every Row, Every Night

How a nightly replica of a vendor ERP stays exact under a hard query timeout, a query-text limit, and no tombstones: the method, shown in the SQL it generates.

Colby Shepherd

I designed and ran a nightly sync that kept a warehouse copy of a vendor ERP exact: roughly 700 tables and 2 billion rows, reconciled every night. The client's code stays with the client, so I rebuilt the method from scratch against a generated stand-in ERP, in the same T-SQL, over a real SQL Server linked server. The generated statements below are real output from that rebuild; the few code excerpts are marked where abridged.

The constraints decide the design

The ERP was reachable only through a linked server backed by the vendor's proprietary driver. Six facts shaped everything that follows:

- **A 15-minute hard timeout** on every query.
- **A limit on query-text length**, which caps how much any one statement can say.
- **Read-only access.** No staging tables, temp tables, or views on the vendor's side.
- **No tombstones.** A deleted row simply disappears.
- **Create and modify dates that cannot be trusted**: sometimes missing, sometimes not updated when a row changes.
- **Undeclared keys.** The real keys are composite, an int plus codes and sometimes a date, and live only in the vendor's data dictionary.

Untrustworthy dates rule out filtering for recent changes, and missing tombstones rule out asking for deletes. What remains is comparing every row, every night, and making that cheap.

One path for every table

Each table runs as its own procedure, started in parallel. The first load, a nightly run, and a full rebuild all take the same seven steps, and every statement sent to the ERP is generated from two things: a copy of the ERP's column metadata, and a hand-kept manifest of each table's key columns.

1	2	3	4	5	6	7
Pull every key and a row hash	**Guard** against a short pull	**Compare** new, changed, deleted	**Prune** to a request list	**Fetch** keys sent in the query text	**Apply** in one transaction	**Reconcile** every count, or roll back

1 Pull every key and a hash of its whole row

One query per table returns each key and an MD5 of the entire row, computed on the ERP's side, so only keys and 32-character hashes cross the wire. In production this single pull finished inside the timeout even at 500 million rows. For a thirteen-column table it looks like this, reformatted from the one line actually sent:

```
DECLARE @d varchar(3) = '$|$', @n char(1) = CHAR(0), @m varchar(max) = '';
SELECT [dunning_id], [dunning_code], [line_code],
       CONVERT(char(32), HASHBYTES('MD5', CONCAT(@m, CONCAT(@m,
           @d, ISNULL(CONVERT(varchar(20), [dunning_id]), @n),
           @d, ISNULL([dunning_code], @n),
           @d, ISNULL([line_code], @n),
           @d, ISNULL(CONVERT(varchar(27), [created_on], 121), @n),
           @d, ISNULL(CONVERT(varchar(27), [modified_on], 121), @n),
```

```
           @d, ISNULL(CONVERT(varchar(40), [hours_001]), @n),
           @d, ISNULL([label_002], @n),
           @d, ISNULL(CONVERT(varchar(20), [ext_id_003]), @n),
           @d, ISNULL(CONVERT(varchar(20), [closed_004]), @n),
           @d, ISNULL(CONVERT(varchar(40), [rate_005]), @n),
           @d, ISNULL(CONVERT(char(10), [start_date_006], 23), @n),
           @d, ISNULL([note_007], @n),
           @d, ISNULL(CONVERT(varchar(40), [balance_008]), @n)))), 2)
FROM dbo.[ar_dunning]
```

742 characters for thirteen columns. The widest stand-in table, 318 columns, needs about 12,300 for the same statement.

Every detail in that statement is there to make the hash honest:

- **Each type gets its own cast.** Default conversions round money to two decimals, cut float to six significant digits, and drop datetime's seconds. The generator picks a style that keeps full precision: 2 for money, 3 for float, 121 for datetime, 23 for date.
- **NULL becomes `CHAR(0)`**, a character no text value contains, so NULL and an empty string hash differently.
- **The delimiter is `$|$`**, not a single pipe, so a pipe moving from one value to its neighbor cannot produce the same string.
- **Every `CONCAT` starts from `@m`, a `varchar(max)`.** Without a large-object argument, SQL Server silently truncates the result at 8,000 characters, or 4,000 once an `nvarchar` column joins in, and a change past that point never reaches the hash. I hit this in production; the rebuild hit it again independently. `CONCAT` takes at most 254 arguments, so wide tables nest one per 126 columns.

The ERP's hash is stored on each replica row, and the next night compares against it. The warehouse never reproduces the ERP's casting, so the two sides never hash the same row differently.

2–4 Guard, compare, prune

A short pull looks exactly like mass deletion, so before comparing anything the procedure checks the count. The replica may shrink by deletes, but only so far:

```
IF @wipe = 0 AND @active_before > 0 AND @pulled < @active_before * (1 - @allowance)
    THROW 50010, @msg, 1;  -- logged, alerted, and not retried
```

Then one pass over the pulled hashes classifies every key. A key the replica lacks is new; a key whose hash differs, or that was deleted and has come back, is changed; a key active in the replica but absent from the pull was deleted in the ERP. Unchanged keys are dropped, and what remains is the request list.

```
UPDATE h SET _action = CASE WHEN t.[dunning_id] IS NULL THEN 'N'
                            WHEN t._is_deleted = 1 OR t._row_hash <> h._row_hash THEN 'C' END
FROM stage.[ar_dunning__hashkey] h
LEFT JOIN replica.[ar_dunning] t
  ON h.[dunning_id] = t.[dunning_id] AND h.[dunning_code] = t.[dunning_code] AND h.[line_code] = t.[line_code];

INSERT INTO stage.[ar_dunning__deleted] ([dunning_id], [dunning_code], [line_code])
SELECT t.[dunning_id], t.[dunning_code], t.[line_code]
FROM replica.[ar_dunning] t
WHERE t._is_deleted = 0
  AND NOT EXISTS (SELECT 1 FROM stage.[ar_dunning__hashkey] h
                  WHERE h.[dunning_id] = t.[dunning_id] AND h.[dunning_code] = t.[dunning_code]
                    AND h.[line_code] = t.[line_code]);
```

Deletes are found by absence: the only way to see one when the source leaves no trace.

5 Send the keys back inside the query text

Nothing can be created on the ERP, so there is nowhere to put a list of keys except the statement itself. Each requested key becomes a literal tuple in a `VALUES` list, joined on every key column:

```
SELECT s.[dunning_id], s.[dunning_code], s.[line_code], s.[created_on], s.[modified_on],
       s.[hours_001], s.[label_002], s.[ext_id_003], s.[closed_004], s.[rate_005],
       s.[start_date_006], s.[note_007], s.[balance_008]
FROM dbo.[ar_dunning] s
JOIN (VALUES (1,'C''3','L002'),
             (1,'C''3','L001'),
             (1,'C''3','L000')) v([dunning_id], [dunning_code], [line_code])
  ON s.[dunning_id] = v.[dunning_id]
 AND s.[dunning_code] = v.[dunning_code]
 AND s.[line_code] = v.[line_code]
```

Three keys shown; a real batch holds hundreds. The code `C'3` goes in as `'C''3'`. Unescaped, the first quoted code would end the string and break the batch.

The batch size is arithmetic on the statement. The select list is the same for every batch of a table, so what remains of the text limit is the room for keys. A running sum of literal lengths closes each batch before it would overflow, with a cap of 5,000 rows:

```
WITH a AS (SELECT _batch, SUM(LEN(_key_literal) + 1)
                  OVER (ORDER BY [dunning_id], [dunning_code], [line_code] ROWS UNBOUNDED PRECEDING) AS running_len
           FROM stage.[ar_dunning__hashkey]),
     b AS (SELECT _batch, (running_len - 1) / (@room - @longest_literal) AS char_batch FROM a),
     c AS (SELECT _batch, char_batch, ROW_NUMBER() OVER (PARTITION BY char_batch ORDER BY char_batch) AS rn FROM b),
     d AS (SELECT _batch, DENSE_RANK() OVER (ORDER BY char_batch, (rn - 1) / @row_cap) AS nb FROM c)
UPDATE d SET _batch = nb;
```

From the procedure, with variables renamed for reading. On the first load, batches landed at up to 14,998 characters against a 15,000 limit. A 318-column table fits about 830 keys per batch; a narrow one about 1,200.

6–7 Apply everything at once, then prove the counts

Fetched rows land in staging. Nothing touches the replica until every batch is in; then one transaction flags the deletes, removes the changed rows, and inserts the fetched ones, each stamped with its load time and the hash from the pull. Before it commits, every count must reconcile, and the replica must hold exactly the keys that were pulled:

```
BEGIN TRAN;
  UPDATE t SET _is_deleted = 1 FROM replica.[ar_dunning] t JOIN stage.[ar_dunning__deleted] d ON …;
  DELETE t FROM replica.[ar_dunning] t JOIN stage.[ar_dunning__hashkey] h ON … WHERE h._action = 'C';
  INSERT INTO replica.[ar_dunning] (/* 13 columns */, _row_hash, _loaded_at, _is_deleted)
  SELECT /* 13 columns */, h._row_hash, @run_at, 0
  FROM stage.[ar_dunning__rows] r JOIN stage.[ar_dunning__hashkey] h ON …;

  IF @flagged <> @deleted OR @removed <> @changed OR @inserted <> @requested OR @active_after <> @pulled
      THROW 50013, @msg, 1;  -- rolls back: the table stays exactly as it was
COMMIT;
```

Abridged: the join conditions are the same three-column key match as above.

That makes failure cheap. Retries are nested, three per batch inside the procedure and three per table in the runner, and a failed `INSERT … EXEC` inserts nothing, so a retry never duplicates a row. A table that fails every attempt is left exactly as it was the night before, and the next run picks up everything it missed. The one email per bad run, sent the way the original's SSIS Send Mail task did, says so:

```
Run 7 (2026-01-07 02:00:00): 119 tables succeeded, 1 did not.

pj_project: stopped by the variance guard after 1 attempt.
  Error 50010: Variance guard: pulled 129344 keys, replica has 136002 active rows, allowance 1.0 percent.
  Not retried. The replica was left untouched. Check the source before the next run.
```

From the failure proof: 5% of a table vanished from the ERP, as in a partial restore. Without the guard, those rows would have been flagged deleted.

The details that decide correctness

Each row below is a one-row change made in the stand-in ERP. An obvious first-draft hash (default conversions, NULL as an empty string, a single pipe, no `varchar(max)`) does not notice any of them. The sync catches every one.

CHANGE	BEFORE → AFTER	FIRST-DRAFT HASH	THIS SYNC
NULL becomes an empty string	`NULL` → `''`	misses it	changed
money, fourth decimal	27563.3800 → 27563.3801	misses it	changed
float, 13th significant digit	1.7917896271428570e+8 → 1.7917896271446490e+8	misses it	changed
datetime, 10 milliseconds	04:37:28.047 → 04:37:28.057	misses it	changed
a pipe moves between neighbors	`'ab'`, `'|cd'` → `'ab|'`, `'cd'`	misses it	changed
a change past the 4,000th character	…xxx → …xxy	misses it	changed

Proving it without the client's system

The stand-in is generated in T-SQL from a seed: 120 tables and 1.2 million rows with undeclared composite keys, quoted codes, untrustworthy dates, and tables up to 318 columns, plus nightly activity that inserts, updates, deletes without a trace, and re-creates deleted keys. The timeout and the read-only login are real, enforced by SQL Server; the text limit is the one constraint simulated, in the single procedure every ERP call passes through.

After every run, an independent audit compares every row of every table with the ERP. It reads the ERP through its own linked server and fingerprints rows with SQL Server's JSON rendering and SHA-256, so a bug in the sync's hashing cannot hide from it. The old process ran beside the new one on the same nights, rebuilt as it was: most tables reloaded in full, the large ones filtered by date and upserted on part of their key.

960 of 960
table audits clean over the first load and seven nights

1,884 rows
fetched on night 7, after comparing all 1,227,989 keys

71,355 rows
wrong in the old process's copy after seven nights, with no errors raised

What I would do differently

- **Close the gap between the pull and the fetch.** The stored hash comes from the pull and the values from the fetch. A row that changes in between, then changes back before the next pull, keeps the in-between values. I never saw transactions land at that hour, but it is possible; fetching each row's hash with its values would close it.
- **Retry only what a retry can fix.** Every failure is retried, including errors that can never succeed on retry.
- **Handle schema changes in place.** A vendor change to a table's columns stops that table, by name, until it is rebuilt from scratch. Adding the column and refetching that one table would keep it running.

The code, the stand-in, and every proof are in data-architecture-case-studies / erp-hashdiff-sync. The statements above come from its T-SQL: sql/warehouse/30_sync_process.sql.